

Reto 2 · Patrones de diseño arquitectónico

Arquitectura de Software

Santiago HM · Simón BU · Sebastián QJ

Índice

1. Actividad 1 · Puntos de dolor
2. Actividad 2.1 · Decisión de patrones
3. Actividad 2.2 · Bitácora frente a la IA
4. Actividad 3.1 · Diseño AS-IS / TO-BE
5. Actividad 3.2 · Tabla de cambio estructural
6. Actividad 3.3 · Fichas de los patrones
7. Actividad 4.1 · Matriz SOLID
8. Actividad 4.2 · Evidencia de comportamiento
9. Actividad 5 · Registro de riesgos
10. Actividad 6 · Vista para el negocio
11. Actividad 6 · Vista para el equipo

Actividad 1 · Puntos de dolor

Reto 2 · Patrones de diseño arquitectónico Equipo: Santiago HM, Simón BU, Sebastián QJ

Qué código se analizó

El AS-IS de este reto es el código que entregamos en el Reto 1, no el original de la hacienda. Vive en:

```
03-src/redisenado/HaciendaNEW/
├─ Bib_Hacienda/      biblioteca de dominio
├─ p_mvcHacienda/     aplicación MVC
```

Todas las referencias de archivo y línea de este documento apuntan ahí. Se verificaron el 5 de septiembre de 2026 con los comandos de la sección 4.

Una advertencia para quien revise el repositorio: la carpeta 05-reto2-patrones/04-src/baseline-input-2026-08-31/ tiene nombre de baseline pero no lo es. Contiene ya aplicados Factory Method, Builder y Observer, más el esqueleto de HistoriaClinica. Si buscas ahí los switches de P-01 o las suscripciones de P-03 no los vas a encontrar, porque ya están resueltos. El AS-IS real es 03-src/redisenado/.

1. Puntos de dolor

Siete encontrados. El enunciado pide cinco, así que en el documento final hay margen para descartar dos.

ID	Dónde	Qué lo hace caro	Cuánto cuesta hoy	Prioridad
P-01	Bib_Hacienda/Clases/Potrero.cs:40-140 (anadir_res) y :202 (agregar) p_mvcHacienda/Servicios/PersistenciaService.cs:512-526 y :571-574	Para dar de alta una res, anadir_res primero traduce el enum del potrero a un string (tipo_vaca, línea 53) y después hace un segundo switch sobre ese texto para ejecutar el new (línea 90). Los dos switches no están atados: si añades un caso al primero y olvidas el segundo, res queda en null, el método sigue y el fallo aparece después dentro de un publisher con el mensaje "Error inesperado en el metodo Informar_Peso_Min", que no dice nada del origen. La misma decisión se repite dos veces en PersistenciaService, y ahí el switch de recarga termina en _ => new Ternero(...): cualquier tipo desconocido guardado en disco vuelve convertido en ternero, en silencio. Aparte, agregar (línea 202) es una segunda puerta al mismo potrero, con las reglas de edad reescritas y una validación de duplicados que anadir_res no tiene.	Escenario: añadir una cuarta categoría de res. 3 clases y 4 archivos: Potrero, PersistenciaService, la clase nueva y Views/Potrero/Create.cshtml. Dentro de ellos, 5 bloques de decisión que hay que sincronizar a mano.	Alta

ID	Dónde	Qué lo hace caro	Cuánto cuesta hoy	Pr io ri d a d
P-02	Eventos/PublisherPesoMin.cs:25-27 Eventos/PublisherPesoVenta.cs:27-29 Eventos/PublisherVacunacionCompletada.cs:30-40 Servicios/ResService.cs:97-99 Views/Res/Index.cshtml:69-71, Views/Venta/Index.cshtml:107-117	Para saber el peso mínimo de una res, seis archivos preguntan <code>res.isTernero</code> / <code>isCebon</code> / <code>isNovillo</code> y eligen la constante a mano. Lo llamativo es que <code>Res</code> ya resolvió bien este problema para el cupo de vacunas: <code>MaxVacunasBacterianas</code> es abstracta y cada subtipo la responde. Peso mínimo, peso de venta y esquema completo no siguieron ese camino. Ninguna de las cadenas tiene rama final, así que si no coincide ningún tipo la variable queda en 0 y el aviso no se dispara: <code>PublisherPesoMin</code> daría por bien alimentada a una res desnutrida sin error visible.	Mismo escenario que P-01. 6 clases y 8 archivos: los 3 publishers, <code>ReglaRes</code> , <code>ReglaVacuna</code> , <code>ResService</code> y 2 vistas. Sumado a P-01: 9 clases y 12 archivos para un solo tipo nuevo.	Alta
P-03	Clases/Potrero.cs:112, 118, 124, 130 Clases/Hacienda.cs:225, 231, 294	Las suscripciones a eventos (+) están escritas dentro del cuerpo de los métodos, sobre publishers que son campos de instancia, y nunca se sueltan. <code>Hacienda</code> está registrada como <code>AddSingleton</code> , así que vive lo que dura el proceso: en la llamada número 200 a <code>alimentar_res</code> se ejecutan 200 handlers, 199 de ellos escribiendo en variables locales de invocaciones ya terminadas. La lista de suscriptores solo crece. Y para saber quién escucha un evento no basta con leer la declaración de la clase, hay que leer el cuerpo de cada método.	Escenario: añadir un aviso nuevo, por ejemplo "res lista para traslado". 8 clases y 8 archivos solo para entender el mecanismo: <code>Hacienda</code> , <code>Potrero</code> y los 6 publishers. 7 puntos de suscripción repartidos en 3 métodos.	Alta
P-04	Clases/FabricadorVacunas.cs:60 y 98 (los dos <code>CrearLote</code>) Interfaces/ICreacionVacuna.cs:13-16 Servicios/VacunaService.cs:26-36	<code>CrearLote</code> está escrito dos veces, idéntico salvo la línea del <code>new</code> y el texto del mensaje. La copia bacteriana arrastra un error que la viva no tiene: en la línea 86 falta el <code>\$</code> de la interpolación, así que el resumen del lote imprime literalmente <code>- Nombre: {nombre}</code> . Nadie lo ha notado porque hay que leer las dos versiones en paralelo para verlo. Las cuatro firmas están además publicadas en <code>ICreacionVacuna</code> , de modo que un tipo nuevo de vacuna obliga a cambiar el contrato y con él todo lo que lo implementa. <code>VacunaService</code> decide qué clase concreta crear mirando cuál de los dos parámetros opcionales vino con valor, que es un tercer sitio donde se toma la misma decisión.	Escenario: incorporar un tercer tipo de vacuna. 7 clases y 11 archivos: la clase nueva, <code>FabricadorVacunas</code> , <code>ICreacionVacuna</code> , <code>Hacienda</code> , <code>VacunaService</code> , <code>VacunaController</code> , <code>PersistenciaService</code> y 4 vistas.	Alta

ID	Dónde	Qué lo hace caro	Cuánto cuesta hoy	Prioridad
P-05	Clases/Res.cs:104-105 Ternero.cs:27,30, Novillo.cs, Cebon.cs enum/ TipoVacuna.cs	Res declara una propiedad abstracta por cada tipo de vacuna que existe, respondida por las tres subclases: 6 propiedades para cubrir 2x3 combinaciones, y cada tipo nuevo de vacuna añade 4 más. El enum TipoVacuna repite la lista de subclases de Vacuna, así que el mismo dato vive en dos sitios que nadie obliga a mantener sincronizados. Si se crea la clase y se olvida el enum, Res.aplicar_vacuna cae en el default de su switch y devuelve un mensaje que no dice cuál era el límite.	Mismo escenario que P-04. 6 clases y 6 archivos: Res, las 3 subclases, TipoVacuna, RegLaVacuna. Sumado a P-04: 17 archivos para un tipo nuevo de vacuna.	Media
P-06	Clases/Autenticacion.cs (150 líneas) Interfaces/IAutenticacion.cs Infrastructure/InterceptorAutenticacion.cs (69 líneas) Servicios/UsuarioService.cs	Hay dos sistemas de usuarios en paralelo y el que tiene las reglas de permisos no está conectado. Autenticacion es la única clase que sabe qué puede hacer cada rol, y nadie la instancia: new Autenticacion no aparece en ningún archivo del proyecto. InterceptorAutenticacion tampoco está registrado, solo se define a sí mismo. Quien autentica de verdad es UsuarioService, que guarda los usuarios en un static List<Usuario> y no comprueba ningún permiso. Son 219 líneas que parecen ser el control de acceso del sistema y no se ejecutan nunca: quien quiera cambiar permisos va a editar el archivo equivocado y no va a notar nada. Aparte, AutorizarOperacion decide el rol comparando el nombre de usuario contra los literales "admin", "empleado" y "visitante", así que un usuario creado desde la aplicación no puede tener permisos, y comunica el resultado lanzando excepción también cuando autoriza.	Escenario: "el veterinario puede aplicar vacunas pero no vender". 5 clases y 6 archivos. Antes de tocar nada hay que leer 219 líneas muertas para descubrir que no corren.	Media
P-07	Clases/Hacienda.cs:182-202 (vender_res) y el vender<T> genérico Clases/InventarioCarnes.cs, InventarioLacteos.cs, InventarioPielles.cs (64 líneas cada una) Servicios/ResService.cs:79	La venta de productos derivados está construida y desconectada. Existen los tres inventarios y un vender<T> genérico, y ninguno se instancia ni se llama desde la aplicación: new InventarioCarnes no aparece en ningún sitio. El único camino de venta vivo es vender_res, el viejo. Las tres clases de inventario son el mismo archivo copiado tres veces, 192 líneas para tres comportamientos idénticos, y ya divergieron en detalles. Los dos formatos de venta conviven dentro de Venta, así que ValidadorVenta tiene que preguntar cuál de los dos trae cada registro antes de saber si es válido.	Escenario: vender un lácteo desde la aplicación. 8 clases y 8 archivos solo para averiguar cuál de los dos caminos está vivo. Más 1 servicio, 1 controlador y 2 vistas para conectarlo.	Media

2. Criterio de priorización

La prioridad no es una impresión. Se calcula así:

archivos que hay que abrir × cercanía al Anexo B

Cercanía vale 2 si una solicitud de cambio del Anexo B toca ese punto de forma directa, 1 si lo toca de lado, 0 si no lo toca.

ID	Archivos	Anexo B	Cercanía	Puntaje	Prioridad
P-01	12 (con P-02)	SC-2 pide colgar un chip de cada res, o sea crear y recargar reses	2	24	Alta
P-02	8	SC-2, mismo motivo	2	16	Alta
P-04	11	SC-3 amplía el registro clínico, y las vacunas son la mitad de ese registro	2	22	Alta

ID	Archivos	Anexo B	Cercanía	Puntaje	Prioridad
P-03	8	SC-2, un chip de geolocalización genera avisos nuevos	2	16	Alta
P-05	6	SC-3, de lado	1	6	Media
P-07	8	SC-1, ya implementada en el Reto 1	1	8	Media
P-06	6	Ninguna solicitud toca permisos	0	0	Media

P-06 queda en media pese a puntuar cero porque 219 líneas muertas que aparentan ser el control de acceso son una trampa para cualquiera que entre al proyecto. Es un costo de lectura, no de cambio, y el criterio numérico no lo captura. Lo dejamos anotado como excepción declarada en vez de forzar la fórmula.

3. Punto que no se interviene

El enunciado exige mínimo uno. Proponemos N-02 como el que va al documento, y dejamos N-01 y N-03 como alternativas por si el equipo prefiere otro.

I D	Dónde	Qué lo hace caro	Cuánto cuesta hoy
N -0 1	<code>Servicios/PersistenciaService.cs</code> (654 líneas, 5 interfaces)	Un solo archivo lee y escribe los seis archivos de <code>Datos/</code> , cada uno con su formato separado por <code> </code> , sin cabecera ni versión. Los métodos de carga llevan try/catch línea por línea, así que una línea corrupta se salta en silencio y el sistema arranca con menos datos de los que hay en disco.	1 clase, 654 líneas, 6 formatos. Intervenirlo costaría unas 12 clases nuevas.
N -0 2	32 sitios en 14 clases (<code>Hacienda 5</code> , <code>Potrero 4</code> , <code>FabricadorVacunas 4</code> , los 3 inventarios 3 c/u, <code>Res 2</code> , <code>Autenticacion 2</code> , los 6 publishers 1 c/u)	Cada método envuelve la excepción del método al que llamó con <code>catch (Exception er) { throw new Exception("Error inesperado en X: " + er.Message); }</code> . Un fallo al añadir una res llega a la vista como "Error inesperado en el metodo anadir_res_potrero: Error inesperado en el metodo anadir_res: ...", con el mensaje real al fondo y la traza original perdida.	14 clases, 32 sitios.
N -0 3	7 vistas: <code>Res/Index.cshtml:69-71</code> , <code>Venta/Index.cshtml:107-117</code> , <code>Potrero/Create.cshtml:46-48</code> , <code>Vacuna/Index.cshtml:76-83</code> , <code>Vacuna/Aplicar.cshtml:67</code> , <code>Vacuna/Create.cshtml:39-46</code> , <code>Res/DetalleVacunas.cshtml:42-49</code>	La etiqueta y el color de cada tipo de res se deciden dentro de la vista, con cadenas de <code>@if (item.Res is Ternero)</code> . Cada tipo nuevo obliga a abrir siete archivos de presentación y acordarse de los siete, sin que nada avise si se olvida uno.	7 vistas de las 22 del proyecto.

Por qué no se interviene N-02

Estos mensajes son la salida observable del sistema: llegan tal cual a las vistas del usuario. La regla 2 del enunciado descuenta 0.5 sobre la nota final por cada cambio no autorizado en el comportamiento observable, así que limpiarlos es exactamente lo que no podemos hacer sin permiso.

El remedio correcto son excepciones de dominio tipadas. Eso toca 14 clases y 32 sitios, reescribe texto congelado y no resuelve ninguna solicitud del Anexo B. Mucho trabajo, riesgo de penalización directa y cero beneficio para el negocio. Lo declaramos como deuda técnica conocida y pedimos autorización, no lo arreglamos este trimestre.

Los otros dos, en corto

N-01 queda fuera porque el alcance excluye "base de datos real" y esta capa se reemplaza entera el día que exista una. Pagaríamos doce clases de indirección sobre algo que se va a tirar. Además, la parte de este

archivo que sí duele, el switch con su `_ => new Ternero(...)`, ya está contada en P-01 y se arregla desde ahí sin tocar las otras 600 líneas.

N-03 queda fuera porque el alcance excluye "interfaz gráfica" y el correo acota el encargo a cómo colaboran los objetos dentro del back. Meter un patrón en la capa de presentación arriesga el tope de 2.5 del criterio 3 por cambio de estilo. El costo real es bajo: son siete etiquetas de texto, y cuando se olvida una el fallo es cosmético y salta a la vista.

4. Cómo se midieron los costos

Los conteos salen de estos comandos, ejecutados sobre `03-src/redisenado/HaciendaNEW/` excluyendo `obj/` y `bin/`. Si en la sustentación los preguntan al azar, se vuelven a correr.

```
# P-01 – sitios que instancian un subtipo de Res
grep -rn "new Ternero\\|new Novillo\\|new Cebon" --include="*.cs" .

# P-02 – sitios que discriminan por subtipo
grep -rn "is Ternero\\|is Novillo\\|is Cebon" --include="*.cs" --include="*.cshtml" .

# P-03 – puntos de suscripción a eventos
grep -rn "evt_.* += " --include="*.cs" .

# P-06 – quién usa realmente la autenticación del dominio
grep -rn "new Autenticacion\\|InterceptorAutenticacion" --include="*.cs" .

# P-07 – quién instancia los inventarios y quién llama a vender
grep -rn "new Inventario\\|vender_res\\|\\.vender(" --include="*.cs" .

# N-02 – excepciones envueltas, por clase
grep -rc "Error inesperado en" --include="*.cs" .
```

Resultados verificados el 5 de septiembre de 2026:

Métrica	Valor
Archivos a abrir para añadir un tipo de res	12 (P-01 + P-02)
Archivos a abrir para añadir un tipo de vacuna	17 (P-04 + P-05)
Puntos de suscripción a eventos dentro de métodos	7, en 3 métodos
Líneas de control de acceso que no se ejecutan nunca	219
Líneas triplicadas en los tres inventarios	192
Excepciones envueltas	32 sitios en 14 clases
Vistas acopladas a clases concretas	7 de 22
Líneas de <code>PersistenciaService</code>	654

5. Pendiente antes de entregar

- ☐ Marcar cuáles tres puntos se encontraron sin IA. El enunciado lo exige y la rúbrica lo audita al azar. La evidencia es la lectura individual previa en `00-lectura-en-frio/`; los que salgan de ahí se registran en la bitácora de la Actividad 2.
- ☐ Elegir cuáles cinco de los siete van al PDF final.
- ☐ Confirmar que N-02 es el punto no intervenido, o cambiarlo por N-01 o N-03.
- ☐ Verificar que cada punto elegido tiene un patrón anclado en la Actividad 2. Un patrón sin P-xx detrás cuesta 0.3 sobre la nota final.

Actividad 2.1 · Decisión de patrones

1. Tabla de decisión

Patrón	Familia	Punto de dolor	Qué gana y qué cuesta	Decisión	Por qué
Factor y Method	Creacional	P-01	Saca la elección del subtipo de los clientes. Cuesta una interfaz, tres creadores y un registro.	Adoptado	<code>Potrero.anadir_res</code> traduce el enum a un string (:53) y después hace un switch sobre ese texto (:90). <code>PersistenciaService</code> repite la decisión en :512–526 y :571–574. Son cuatro sitios que hay que sincronizar a mano.
Abstract Factory	Creacional	P-01	Coordinaría familias de productos. Cuesta una interfaz de fábrica más por familia.	Descartado	<code>Res</code> y <code>Vacuna</code> varían por separado. No existe la regla "un Ternero solo admite la vacuna X de la misma familia", así que la fábrica abstracta protegería una restricción que el dominio no tiene. El propio enunciado advierte que con una sola familia no se justifica.
Builder	Creacional	P-04	Separa la secuencia común de lo que cambia por variante. Cuesta una interfaz de builder y una clase por variante.	Adoptado	<code>CrearLote</code> está escrito dos veces (<code>FabricadorVacunas.cs:60</code> y :98), idéntico salvo el <code>new</code> y el texto. La copia bacteriana arrastra un \$ faltante en :86 que la viva no tiene. Duplicar el cuerpo ya produjo un defecto real.
Prototype	Creacional	P-04	Clonaría la vacuna base del lote. Cuesta un contrato de clonación y decidir la profundidad de la copia.	Descartado	Resuelve solo el bucle del lote. Deja intactas las cuatro firmas del contrato y la selección de variante en <code>VacunaService</code> , que son las otras dos terceras partes de P-04. Builder cubre las tres.
Singleton	Creacional	P-03	Darí­a un punto de acceso único a <code>Hacienda</code> . Cuesta estado global y testabilidad.	Descartado	<code>Hacienda</code> ya vive como singleton, pero por <code>AddSingleton</code> en la raíz de composición, que es una decisión de la aplicación y se puede cambiar sin tocar el dominio. Un <code>Hacienda.Instance</code> metería esa decisión dentro del dominio y ningún consumidor podría sustituirla en una prueba.
Adapter	Estructural	P-07	Deja que el camino genérico de venta acepte un potrero. Cuesta una clase de traducción y la conversión de tipo.	Adoptado	<code>Hacienda.vender</code> pide <code>IInventario<Producto></code> y <code>Potrero</code> implementa <code>IInventario<Res></code> . Como <code>IInventario<T></code> es invariante, el compilador no acepta el segundo donde se espera el primero. Sin adaptador quedan dos caminos de venta y hay que mantener los dos.
Facade	Estructural	P-07	Una entrada única a la coordinación. Cuesta una capa más y el riesgo de absorber lógica.	Descartado	<code>Hacienda</code> ya hace ese papel: <code>anadir_res_potrero</code> (:125) solo busca el potrero y delega. Añadir otra fachada duplicaría el rol, y presentar la que ya existe como incorporación nuestra sería atribuirnos una estructura del Reto 1. El enunciado además avisa de que Facade tiende a romper SRP absorbiendo negocio.
Decorator	Estructural	P-06	Envolvería cada operación con su chequeo de permisos. Cuesta un decorador por operación.	Descartado	Activar permisos hoy haría fallar operaciones que hoy pasan, y eso es un cambio de comportamiento observable no autorizado: 0.5 sobre la nota final por caso. El patrón no resuelve la decisión de fondo, que es si el negocio quiere empezar a denegar acciones.
Proxy	Estructural	P-06	Intercepta llamadas para autorizar. Cuesta una capa de intercepción.	Descartado	Ya está la técnica en el proyecto: <code>p_mvchacienda/Program.cs:41–49</code> usa <code>Castle.DynamicProxy</code> para envolver los validadores. Presentarlo como patrón nuevo sería contar algo que ya estaba, y choca con la misma barrera de comportamiento que Decorator.

Patrón	Familia	Punto de dolor	Qué gana y qué cuesta	Decisión	Por qué
Observer	Comportamiento	P-03	Fija quién escucha y por cuánto tiempo. Cuesta una interfaz de observador y un objeto que recolecta.	Adoptado	Hay siete <code>+=</code> dentro de cuerpos de método (<code>Potrero.cs:112,118,124,130</code> y <code>Hacienda.cs:225,231,294</code>) que nunca se sueltan. Con <code>Hacienda</code> como singleton, la llamada número 200 ejecuta 200 handlers.
Strategy	Comportamiento	P-02	Extrae las reglas por tipo a objetos. Cuesta una interfaz, tres estrategias y decidir quién las asigna.	Descartado	La variación es por subtipo de <code>Res</code> , y la herencia ya la representa. <code>Res.cs:104-105</code> lo demuestra: <code>MaxVacunasBacterianas</code> es abstracta y cada subtipo responde. P-02 no es que falte un mecanismo, es que tres publishers no usaron el que ya existía. Se arregla subiendo esas reglas a <code>Res</code> , no añadiendo una jerarquía paralela.
Template Method	Comportamiento	P-04	Fija la secuencia en una clase base y deja huecos a las subclasses. Cuesta herencia obligatoria.	Descartado	La secuencia de lote y la individual comparten poco: la de lote numera, omite duplicados y cuenta. Forzar una plantilla común dejaría pasos vacíos en una de las dos ramas, que es justo el error de LSP que el enunciado nombra. Builder consigue lo mismo por composición.
Visitor	Comportamiento	P-05	Abarata añadir operaciones sobre la jerarquía. Encarece añadir tipos.	Descartado	Optimiza el eje contrario al que duele. P-05 es caro precisamente cuando se agrega un tipo nuevo de vacuna, y Visitor obliga a tocar todos los visitantes cada vez que aparece un elemento nuevo.
Chain of Responsibility	Comportamiento	P-06	Encadena chequeos de autorización. Cuesta la cadena y su orden.	Descartado	Mismo bloqueo que Decorator y Proxy: conectar la autorización cambia comportamiento observable. Además, la cadena resuelve el orden de varios chequeos, y aquí no hay varios chequeos, hay uno que nadie ejecuta.

Catorce evaluados: cinco creacionales, cuatro estructurales, cinco de comportamiento. Cuatro adoptados, diez descartados.

2. Los cuatro adoptados

Factory Method — P-01

En el AS-IS, dar de alta una res pasa por dos switches encadenados dentro de `Potrero.anadir_res`, y la misma decisión se repite dos veces más en `PersistenciaService`. El de recarga termina en `_ => new Ternero(...)`, así que un tipo desconocido guardado en disco vuelve convertido en ternero sin avisar.

En el TO-BE la decisión desaparece. `ICreadorRes` declara `AplicaA(ushort edad)` y `CatalogoCreadoresRes` resuelve por capacidad, no por condicional:

```
public ICreadorRes ParaEdad(ushort edad)
{
    ICreadorRes creador = creadores.FirstOrDefault(c => c.AplicaA(edad));
    ...
}
```

Una categoría nueva se registra y ya. Ningún cliente cambia.

Lo que cuesta: cuatro clases nuevas y un salto de indirección. Quien lee `ParaEdad` no ve qué subtipo sale, tiene que ir al creador. Aceptamos ese costo porque el alternativo es mantener cuatro switches sincronizados a mano.

El riesgo conocido es terminar con una fábrica que crece con un `case` por tipo, que es el error que el enunciado nombra primero. Por eso el catálogo pregunta a cada creador si aplica en vez de decidir él.

Builder — P-04

`FabricadorVacunas` pasa a `Director`. Conoce la secuencia común, validar, construir, registrar y resumir, pero no qué variante se está creando. Cada variante vive en su `IVacunaBuilder`.

Lo interesante del caso es el \$ faltante de la línea 86. Al unificar las dos copias había que decidir qué hacer con él, y la salida del sistema está congelada. La solución fue declararlo en el contrato del builder:

```
// Nombre tal como debe aparecer en el resumen de lote. Existe porque las
// dos variantes no lo imprimen igual y ese texto es observable.
string NombreEnResumenDeLote(string nombre);
```

El defecto se conserva a propósito, y ahora está documentado en vez de escondido en una copia.

Lo que cuesta, y esto queda declarado como deuda: `ICreacionVacuna` sigue publicando cuatro firmas. Builder eliminó el cuerpo duplicado, no el contrato. Un tercer tipo de vacuna todavía obliga a tocar esa interfaz. Resolverlo entra en la deuda que dejamos por escrito, no en este trimestre.

Observer — P-03

Las siete suscripciones dentro de métodos se reducen a tres, y suben al constructor de `Hacienda`:

```
publisher_peso_min.evt_peso_min += recolectorMensajes.Recibir;
publisher_peso_ideal.evt_peso_venta += recolectorMensajes.Recibir;
publisher_vacunacion_completa.evt_vacunacion_completada += recolectorMensajes.Recibir;
```

`RecolectorMensajes` es el único observador y solo guarda avisos mientras una operación tiene una captura abierta. Fuera de una captura los descarta, que es exactamente lo que pasaba antes cuando el publisher no tenía suscriptores. Así el orden de los mensajes se conserva y ninguna operación lee los de otra.

Lo que cuesta: dos abstracciones nuevas y un mecanismo de captura que hay que entender antes de tocar un publisher. A cambio, quién escucha qué se lee en un solo sitio.

Adapter — P-07

Este es el más pequeño y el que más discutimos, porque nació de un problema de compilación, no de diseño.

`Hacienda.vender` opera sobre `IInventario<Producto>`. `Potrero` implementa `IInventario<Res>`. En C# los genéricos son invariantes cuando el parámetro aparece en posición de entrada y de salida, y `IInventario<T>` tiene `agregar(T)` y `retirar(T)`, así que un `IInventario<Res>` no puede pasar por `IInventario<Producto>` aunque `Res` herede de `Producto`.

Sin adaptador quedan dos caminos de venta vivos y hay que mantener los dos. `InventarioPotrero` traduce, y la venta de una res pasa por el mismo método genérico que cualquier otro producto:

```
string mensaje = _hacienda.vender(new InventarioPotrero(potrero), res, monto);
```

Vive en `p_mvchacienda/Servicios/`, no en la biblioteca. Esa ubicación es deliberada: la incompatibilidad es de tipos, no de negocio, y resolverla en el dominio habría obligado a cambiar `IInventario<T>`.

Lo que cuesta: la conversión de `Producto` a `Res` se verifica en tiempo de ejecución, no de compilación. Si alguien le pasa un lácteo, `InventarioPotrero` lanza. Es un chequeo que el compilador hacía antes por nosotros y ahora hacemos a mano.

Actividad 2.2 · Bitácora frente a la IA

ID	Qué consultamos	Qué propuso la herramienta	Qué hicimos	Argumento propio y evidencia
B-01	Auditar la implementación de SC-3	Reportó que <code>EventoClinico</code> no tiene productor: <code>RegistrarEvento</code> o solo se llama desde el arnés de pruebas	Aceptamos	Comprobado con <code>grep -rn "RegistrarEvento"</code> : una sola llamada, en <code>HaciendaReto2.Verification/Program.cs:702</code> . Sin un formulario que cree eventos, la historia clínica era la lista de vacunas de siempre movida de sitio.
B-02	Si aplicar una vacuna debe generar además un evento clínico automático	Propuso que sí, para leer la historia como una línea de tiempo única	Rechazamos	Una vacuna ya tiene tipo, lote y dos fechas. Aplanarla en un evento genérico de tres campos pierde información y deja el mismo hecho registrado dos veces. Además SC-3 dice "además de las vacunas", o sea trata las vacunas como algo que ya existe y la historia como lo que se suma.
B-03	Por qué <code>p_mvchacienda</code> no compila	Detectó 22 errores en la ruta de ventas	Aceptamos el diagnóstico, decidimos nosotros el alcance	Le pedimos que compilara también la copia intacta del baseline: los mismos 22 errores. Son previos a nuestro trabajo. Decidimos arreglarlos solo dentro de <code>p_mvchacienda</code> , sin tocar <code>Bib_Hacienda</code> .
B-04	Cómo vender una res si <code>Hacienda.vender</code> pide <code>IInventario<Producto></code> y <code>Potrero</code> es <code>IInventario<Res></code>	Propuso un Adapter	Aceptamos, con la ubicación puesta por nosotros	<code>IInventario<T></code> tiene <code>agregar(T)</code> y <code>retirar(T)</code> , así que es invariante y la conversión no compila. El adaptador va en <code>p_mvchacienda/Servicios/InventarioPotrero.cs</code> y no en la biblioteca: la incompatibilidad es de tipos, no de negocio.
B-05	Dónde está el AS-IS para analizar los puntos de dolor	Asumió que era <code>05-retro2-patrones/04-src/baseline-input-2026-08-31/</code>	Corregimos	Esa carpeta ya tiene <code>Clases/Creacion/</code> y <code>Clases/Construccion/</code> , o sea Factory Method y Builder aplicados, más <code>RecolectorMensajes</code> . No es un baseline. El AS-IS real es <code>03-src/rediseñado/HaciendaNEW/</code> , donde <code>Creacion/</code> no existe. Si no lo detectamos, la mitad de A1 apuntaba a código inexistente.
B-06	Verificar las referencias de archivo y línea de A1	Encontró que nuestra versión anterior decía que <code>Hacienda.anadir_res_potrero</code> decide entre subtipos	Corregimos un error nuestro	<code>Hacienda.cs:125-137</code> solo busca el potrero y delega. El switch está en <code>Potrero.anadir_res</code> : arma el string en <code>:53</code> y decide en <code>:90</code> . La afirmación anterior no habría resistido la pregunta directa en sustentación.
B-07	Cómo priorizar los puntos de dolor con un criterio defendible	Propuso archivos que hay que abrir × cercanía al Anexo B	Aceptamos, con una excepción declarada	El criterio funciona para seis de los siete. P-06 puntúa cero porque ninguna solicitud toca permisos, y aun así lo dejamos en media: 219 líneas muertas que aparentan ser el control de acceso son costo de lectura, y la fórmula solo mide costo de cambio. Preferimos declarar la excepción antes que forzar el número.

ID	Qué consultamos	Qué propuso la herramienta	Qué hicimos	Argumento propio y evidencia
B-8	Revisar la combinación de patrones que traíamos del borrador: SC-3 con Strategy, Observer y Chain of Responsibility	Sostuvo la propuesta inicial	Corregimos nuestra propia decisión	Strategy cae porque Res.cs:104-105 ya resuelve la variación por subtipo con una propiedad abstracta. P-02 no es que falte el mecanismo, es que tres publishers no usaron el que había. Chain of Responsibility cae porque conectar la autorización haría fallar operaciones que hoy pasan, y eso son 0.5 sobre la nota por caso. Quedaron Factory Method, Builder, Observer y Adapter.
B-9	Si el Adapter cuenta como patrón adoptado o solo como arreglo de compilación	Lo trató como arreglo	Corregimos	Está en el código entregado. Un patrón que existe en el código y no aparece en el diagrama rompe la correspondencia, y la regla 5 del enunciado topa los criterios 3 y 4 en 3.0 por eso. Lo declaramos en A2 y va al TO-BE.
B-10	Dónde declarar la persistencia de los eventos clínicos	Propuso añadir los métodos a IPersistencia de la biblioteca	Corregimos	Habría obligado a tocar Bib_Hacienda, que estaba fuera de alcance por decisión del equipo. Creamos IPersistenciaEventosClinicos dentro de p_mvchacienda/Servicios/ y PersistenciaService lo implementa junto a los cinco puertos que ya tenía.

Actividad 3.1 · Diseño AS-IS / TO-BE

El artefacto canónico es `diagramas/A3-ASIS-T0BE-LAYERED.drawio`: una sola página con dos capas Draw.io reales, AS-IS y T0-BE.

Cómo se lee

Apagando la capa T0-BE queda el recorte del diseño actual, con lo que sale o cambia de responsabilidad marcado en rojo punteado. Encendiendo las dos aparece el diseño futuro completo: los elementos conservados no se mueven de sitio, el T0-BE solo añade participantes y relaciones en el espacio libre.

Las dos vistas están exportadas como `A3-ASIS.png/svg` y `A3-T0BE.png/svg`. Los archivos llevan el XML incrustado, así que se reabren editables en Draw.io.

De dónde sale cada capa

Capa	Código
AS-IS	<code>03-src/redisenado/HaciendaNEW/</code> — lo que entregamos en el Reto 1
TO-BE	<code>05-reto2-patrones/04-src/active/</code> — lo que entregamos ahora

Cada caja lleva la referencia de archivo y línea con la que se puede verificar. Nada del diagrama está puesto de memoria.

Código de color

El color dice a qué patrón pertenece cada clase, que es lo que pide el enunciado. Negro es lo que se conserva sin cambios.

Color	Significado
Negro	Se conserva sin cambios
Rojo punteado	Sale o cambia de responsabilidad
Azul	Factory Method (P-01)
Verde	Builder (P-04)
Amarillo	Observer (P-03)
Naranja	Adapter (P-07)
Morado	SC-3, historia clínica (sin patrón asociado)

`FabricadorVacunas` aparece dos veces a propósito: en rojo dentro del AS-IS, con las dos copias de `CrearLote` y el `$` que falta en la línea 86; y en verde dentro del TO-BE, ya como Director del Builder. Es el caso más claro de "se transforma" y separarlo deja ver qué salió y qué entró.

`IInventario<Producto>` va en negro. Esa interfaz existe igual antes y después; el papel de Target lo comunica la flecha de `InventarioPotrero`, no el color. `Producto` aparece en el diagrama porque sin él no se entiende el Adapter: `Res` hereda de `Producto`, pero `IInventario<Res>` no puede pasar por `IInventario<Producto>` porque el genérico es invariante.

Tres publishers salen en gris punteado. Se instancian y nadie los escucha: `PublisherPotreroMitad` y `PublisherPotreroLleno` en `active/Potrero.cs:23-24`, y `PublisherVacunaVencida` en `active/Hacienda.cs:46`. Los avisos que emiten no llegan a ningún observador. Está dibujado así porque es lo que hace el código, no porque convenga.

Qué se verificó antes de exportar

El enrutado trata las cajas de las dos capas como obstáculo para las aristas de las dos capas, así que ninguna caja del TO-BE tapa una línea del AS-IS ni al revés.

```
39 aristas · 41 cajas · 0 cruce sobre caja · 0 puerto compartido
                0 líneas montadas · 0 ruta indefinida
```

```
aristas de una capa sobre cajas de la otra: 0
cajas del TO-BE encima de cajas del AS-IS: 0
```

Aparte del enrutado, un segundo script recorre el `.drawio` y comprueba que cada título de caja corresponde a un `class`, `interface` o `enum` real, y que cada referencia `Archivo.cs:NNN` apunta a la línea que dice. Las 39 cajas y las 17 referencias resuelven. Ese script se puede volver a correr cuando el código cambie.

Que el TO-BE no tape nada no es un detalle estético. Si una caja nueva cubriera una relación del AS-IS, al encender ambas capas se perdería justo lo que el lector quiere comparar.

Restricciones observables que el diseño conserva

- Builder mantiene el tipo de vacuna, la construcción individual y por lote, y los mensajes existentes. Incluido el texto literal de `FabricadorVacunas.cs:86`, que `BuilderBacteriana` conserva devolviendo `"{nombre}"` sin interpolar.
- Observer conserva el orden de los avisos: en `Potrero`, mitad, lleno, peso mínimo, peso venta; en `Hacienda`, peso mínimo, peso venta.
- Factory Method resuelve por `AplicaA(edad)`, no por un condicional dentro de la fábrica. Una categoría nueva se registra en la raíz de composición y ningún cliente cambia.
- Adapter vive en `p_mvcHacienda/Servicios/`, no en la biblioteca. La incompatibilidad que resuelve es de tipos, no de negocio.

Actividad 3.2 · Tabla de cambio estructural

Una fila por elemento. De cada uno se dice qué pasó con él y cómo se reconectó lo que dependía de él.

AS-IS: 03-src/redisenado/HaciendaNEW/. TO-BE: 05-reto2-patrones/04-src/active/.

Factory Method — P-01

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-01	Potrero.anadir_res	Se transforma	Traducía el enum a tipo_vaca (:53) y decidía el subtipo con un segundo switch (:90-101)	Recibe una Res ya creada	Hacienda.anadir_res_potrero seguía delegando igual. La creación pasa al catálogo
E-02	ICreadorRes	Entra	—	Declara Categoria, AplicaA(edad) y Crear(nombre,peso,edad)	Nadie dependía de él. Lo consumen el catálogo y la raíz de composición
E-03	CreadorTernero, CreadorCebon, CreadorNovillo	Entra	—	Cada uno conoce una sola categoría y responde si aplica a una edad	Sustituyen los case del switch de E-01
E-04	CatalogoCreadoresRes	Entra	—	ParaEdad(edad) resuelve por capacidad declarada, sin condicional	Program lo registra; PersistenciaService lo usa para reconstruir
E-05	PersistenciaService (switches de res)	Se transforma	Decidía el subtipo en :512-526 y en :571-574, con _ => new Ternero(...) como respaldo silencioso	Pide el creador al catálogo	El respaldo legacy se conserva: si la categoría no existe, cae en Ternero igual que antes

Builder — P-04

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-06	Fabricador Vacunas	Se transforma	Dos copias de CrearLote (:60 y :98), idénticas salvo el new y el texto	Director: conoce la secuencia común y no la variante	Hacienda sigue llamando las mismas cuatro sobrecargas, que ahora delegan
E-07	IVacunaBuilder	Entra	—	Declara lo que distingue a cada variante, incluido NombreEnResumenDeLote	Lo consume el Director
E-08	BuilderBacteriana, BuilderViviana	Entra	—	Aportan el dato propio y construyen la vacuna	Reemplazan el cuerpo duplicado de E-06
E-10	Literal de Fabricador Vacunas.cs: 86	Se transforma	Imprimía — Nombre: {nombre} por un \$ faltante	Igual, ahora declarado	BuilderBacteriana.NombreEnResumenDeLote devuelve "{nombre}" a propósito. Salida observable congelada

Observer — P-03

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-11	Suscripciones <code>+=</code> dentro de métodos	Sale	Siete puntos: <code>Potrero.cs:112,118,124,130</code> y <code>Hacienda.cs:225,231,294</code> , nunca liberados	—	Se sustituyen por tres suscripciones en el constructor de <code>Hacienda (:84-86)</code>
E-12	<code>IobservadorMensaje</code>	Entra	—	Un solo rol: <code>Recibir(mensaje)</code> , porque los seis publishers emiten un <code>string</code>	Lo implementa el recolector
E-13	<code>RecolectorMensajes</code>	Entra	—	Único observador. Guarda avisos solo mientras hay una captura abierta	Los publishers notifican a él en vez de a lambdas locales
E-14	<code>CapturaMensajes</code>	Entra	—	Delimita qué operación lee qué mensajes	Cada operación abre la suya, así ninguna lee los de otra
E-15	<code>PublisherPesoMin</code> , <code>PublisherPesoVenta</code> , <code>PublisherVacunacionCompletada</code>	Se transforma	Emitían a lambdas locales	Emiten al recolector	Suscritos en <code>Hacienda.cs:84-86</code> . El orden de emisión no cambia
E-15b	<code>PublisherPotreroMita</code> , <code>PublisherPotreroLleno</code>	Se transforma	Sus avisos llegaban a las lambdas de <code>Potrero.anadir_res</code>	Se instancian en <code>active/Potrero.cs:23-24</code> y nadie los escucha	Nadie. Sus avisos se pierden, igual que antes cuando el publisher no tenía suscriptores
E-15c	<code>PublisherVacunaVencida</code>	Se transforma	Ya estaba sin uso	Se instancia en <code>active/Hacienda.cs:46</code> y nunca se suscribe ni se invoca	Nadie. Deuda declarada

Adapter — P-07

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-16	<code>Hacienda.vender_res</code>	Sale	Único camino de venta vivo (<code>Hacienda.cs:182-202</code>)	—	<code>ResService.Vender</code> pasa al camino genérico
E-17	<code>InventarioPotrero</code>	Entra	—	Traduce <code>IInventario<Res></code> a <code>IInventario<Producto></code>	<code>ResService</code> envuelve el potrero antes de llamar a <code>vender</code>
E-19	<code>ResService.Vender</code>	Se transforma	Llamaba a <code>vender_res</code>	Llama a <code>_hacienda.vender(new InventarioPotrero(potrero), res, monto)</code>	El controlador no cambia

SC-3 — historia clínica

Sin patrón asociado. Es composición de dominio, y ponerle un patrón encima sería sobreingeniería.

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-20	Res.l_vacunas_aplicadas	Sale	Campo <code>List<Vacuna></code> dentro de Res	—	Su contenido pasa a <code>HistoriaClinica</code>
E-21	HistoriaClinica	Entra	No existía en el AS-IS	Dueña de dos colecciones: <code>VacunasAplicadas</code> y <code>EventosClinicos</code>	<code>Res.L_vacunas_aplicadas</code> delega en ella y conserva el aliasing anterior
E-22	EventoClinico	Entra	—	Registro inmutable de fecha, concepto y observación	Lo crean el formulario de la vista y la carga de persistencia
E-23	IPersistenciaEventosClinicos	Entra	—	Puerto de guardado y carga de eventos	Vive en <code>p_mvchacienda/Servicios/</code> para no tocar la biblioteca. <code>PersistenciaService</code> lo implementa junto a los cinco puertos que ya tenía
E-24	ResControler	Se transforma	Solo <code>DetalleVacunas</code>	Añade <code>HistoriaClinica</code> (GET) y <code>RegistrarEvento</code> (POST)	La vista <code>Views/Res/HistoriaClinica.cshtml</code> muestra vacunas y eventos como una sola historia
E-25	Program	Se transforma	Cargaba potreros, reses, vacunas aplicadas y ventas	Añade <code>CargarEventosClinicos</code> junto a <code>CargarVacunasAplicadas</code>	Sin esa línea la historia no sobrevive a un reinicio

Cambios de contrato declarados

SC-3 endureció cuatro puntos que antes aceptaban cualquier cosa. Van declarados aquí porque el comportamiento observable está congelado y estos son los únicos cambios autorizados por la solicitud.

Punto	Antes	Ahora
<code>res.L_vacunas_aplicadas = null</code>	Asignaba sin fallar	<code>ArgumentNullException</code>
Asignar una lista con duplicados	Asignaba sin fallar	<code>ArgumentException</code>
<code>new Res(..., historiaClinica: null)</code>	Aceptaba, el campo quedaba muerto	<code>ArgumentNullException</code>
Compartir una historia entre dos reses	Aceptaba en silencio	<code>InvalidOperationException</code>

Dos elementos quedan fuera de la tabla porque no cambian y esta tabla es de cambios. `ICreacionVacuna` sigue publicando sus cuatro firmas: Builder quitó el cuerpo duplicado, no el contrato, y eso queda como deuda declarada. `IInventario<Producto>` es byte a byte el mismo archivo; lo que cambia es que ahora tiene un implementador que acepta potreros.

Sobre los publishers sin observador: el Observer del TO-BE cubre los tres avisos que `Hacienda` emite. Los cuatro publishers que `Potrero` instancia y `publisher_vacuna_vencida` siguen sin suscriptor. No los conectamos porque hacerlo añadiría mensajes a la salida y eso es un cambio de comportamiento observable no autorizado.

Sobre `HistoriaClinica`: en el AS-IS (03-src/redisenado/) la clase no existe. Sí aparece, vacía y sin uso, en 05-reto2-patronos/04-src/baseline-input-2026-08-31/, que es un snapshot intermedio y no el punto de partida de este reto.

Actividad 3.3 · Fichas de los patrones

AS-IS: 03-src/redisenado/HaciendaNEW/. TO-BE: 05-reto2-patrones/04-src/active/.

Ficha 1 — Factory Method

Campo	Contenido
Patrón y punto de dolor que resuelve	Factory Method, responde a P-01. <code>Potrero.anadir_res</code> arma un string en <code>Potrero.cs:53</code> y decide el subtipo con un switch en <code>Potrero.cs:90-101</code> . La misma decisión se repite en <code>PersistenciaService.cs:512-526</code> y <code>:571-574</code> , esta última con <code>_ => new Ternero(...)</code> : un tipo desconocido en disco vuelve convertido en ternero sin avisar.
Alternativas que evaluaron	No hacer nada: mantener los cuatro switches sincronizados a mano. Se descarta porque añadir una categoría obliga a abrir 4 archivos y nada avisa si se olvida uno. Abstract Factory: descartada, <code>Res</code> y <code>Vacuna</code> varían por separado y no existe la regla de que un subtipo de res exija una familia de vacunas. Fábrica única con switch: descartada, mueve el condicional de sitio en vez de eliminarlo, que es el error que el enunciado nombra primero.
Qué sale y qué entra	Sale: el switch de <code>Potrero.anadir_res</code> y los dos de <code>PersistenciaService</code> . Entran: <code>ICreadorRes</code> (<code>Creator</code>), <code>CreadorTernero</code> , <code>CreadorCebon</code> y <code>CreadorNovillo</code> (<code>ConcreteCreator</code>), y <code>CatalogoCreadoresRes</code> (<code>Registry</code> , resuelve por <code>AplicaA(edad)</code> en <code>CatalogoCreadoresRes.cs:38</code>).
Cómo se relaciona	Los construye <code>p_mvchacienda/Program.cs:74-78</code> ; los consumidores sin contenedor usan <code>CatalogoCreadoresRes.PorDefecto()</code> . Los usan <code>Hacienda.anadir_res_potrero</code> (<code>Hacienda.cs:178</code>) y <code>PersistenciaService</code> (<code>:192</code> , <code>:602</code> , <code>:648</code>). Con Adapter: el potrero que envuelve <code>InventarioPotrero</code> contiene reses creadas por estos creadores.
Impacto	Creadas 5, modificadas 4 (<code>Hacienda</code> , <code>Potrero</code> , <code>PersistenciaService</code> , <code>Program</code>), eliminadas 0. Anexo B: SC-2 crea reses con chip por el mismo catálogo, sin tocar clientes.
Qué cuesta	5 clases más y un salto de indirección: quien lee <code>ParaEdad</code> no ve qué subtipo sale, tiene que abrir el creador. Depurar el alta de una res pasa de un switch a tres saltos.
Origen	Idea propia. B-10 registra la corrección: el borrador traía Strategy y Chain of Responsibility, y los descartamos al ver el código.

Ficha 2 — Builder

Campo	Contenido
Patrón y punto de dolor que resuelve	Builder, responde a P-04. <code>CrearLote</code> está escrito dos veces en <code>FabricadorVacunas.cs:60</code> y <code>:98</code> , idénticas salvo el <code>new</code> y el texto. La copia bacteriana arrastra un <code>\$</code> faltante en <code>FabricadorVacunas.cs:86</code> que la viva no tiene: el resumen imprime literalmente <code>- Nombre: {nombre}</code> .
Alternativas que evaluaron	No hacer nada: dejar las dos copias. Se descarta porque duplicar el cuerpo ya produjo un defecto real que nadie detectó. Prototype: descartada, resuelve el bucle del lote pero deja intactas las cuatro firmas del contrato y la selección de variante en <code>VacunaService</code> . Template Method: descartada, la secuencia de lote numera, omite duplicados y cuenta, y la individual no hace nada de eso; forzar una plantilla común deja pasos vacíos en una rama.
Qué sale y qué entra	Sale: el cuerpo duplicado de los dos <code>CrearLote</code> . Entran: <code>IVacunaBuilder</code> (<code>Builder</code>), <code>BuilderBacteriana</code> y <code>BuilderViva</code> (<code>ConcreteBuilder</code>). <code>FabricadorVacunas</code> se transforma en <code>Director</code> y conserva la secuencia común en <code>FabricadorVacunas.cs:51</code> y <code>:71</code> .
Cómo se relaciona	Los construye <code>FabricadorVacunas</code> en sus cuatro sobrecargas de compatibilidad (<code>:28</code> , <code>:34</code> , <code>:40</code> , <code>:46</code>). Los usa el propio <code>Director</code> . No interactúa con los otros tres patrones adoptados.
Impacto	Creadas 3, modificadas 1 (<code>FabricadorVacunas</code>), eliminadas 0. Anexo B: SC-3 registra vacunas en la historia clínica y un tipo nuevo de vacuna reutiliza el <code>Director</code> sin modificarlo.

Campo	Contenido
Qué cuesta	3 clases más y un contrato nuevo que leer antes de tocar la creación de vacunas. <code>ICreacionVacuna</code> sigue publicando cuatro firmas: el patrón quitó el cuerpo duplicado, no el contrato. Queda declarado como deuda.
Origen	Idea propia. B-09 registra la decisión de conservar el \$ faltante declarándolo en <code>IVacunaBuilder.NombreEnResumenDeLote</code> en vez de corregirlo.

Ficha 3 — Observer

Campo	Contenido
Patrón y punto de dolor que resuelve	Observer, responde a P-03. Hay siete suscripciones <code>+=</code> dentro de cuerpos de método que nunca se liberan: <code>Potrero.cs:112, :118, :124, :130</code> y <code>Hacienda.cs:225, :231, :294</code> . Con <code>Hacienda</code> registrada como singleton, la llamada número 200 a <code>alimentar_res</code> ejecuta 200 handlers.
Alternativas que evaluaron	No hacer nada: dejar las suscripciones donde están. Se descarta porque la lista de suscriptores solo crece mientras viva el proceso, y para saber quién escucha un evento hay que leer el cuerpo de cada método. Desuscribir al final de cada operación: descartada, exige un <code>-=</code> por cada <code>+=</code> y un <code>try/finally</code> , y basta olvidar uno para volver al mismo punto. Singleton como bus de eventos: descartada, mete estado global en el dominio y ningún consumidor podría sustituirlo en una prueba.
Qué sale y qué entra	Salen: las siete suscripciones dentro de métodos. Entran: <code>IObservadorMensaje</code> (Observer), <code>RecolectorMensajes</code> (ConcreteObserver) y <code>CapturaMensajes</code> , que delimita qué operación lee qué avisos. Las suscripciones bajan a tres, en el constructor de <code>Hacienda.cs:84-86</code> .
Cómo se relaciona	Lo construye <code>Hacienda.cs:52</code> como campo. Lo usan los tres publishers de <code>Hacienda</code> , y cada operación abre su <code>Capturar()</code> . No interactúa con los otros tres patrones. Los cuatro publishers de <code>Potrero</code> y <code>publisher_vacuna_vencida</code> (<code>Hacienda.cs:46</code>) quedan sin observador: conectarlos añadiría mensajes a la salida y eso es un cambio observable no autorizado.
Impacto	Creadas 3, modificadas 2 (<code>Hacienda</code> , <code>Potrero</code>), eliminadas 0. Anexo B: SC-2 añade un aviso de geolocalización suscribiendo al recolector, sin tocar <code>Hacienda</code> .
Qué cuesta	3 clases más y un mecanismo de captura que hay que entender antes de tocar un publisher. Un aviso emitido fuera de una captura se descarta en silencio, lo que hace más difícil depurar por qué un mensaje no aparece.
Origen	Idea propia, confirmada en B-10 al revisar la combinación del borrador.

Ficha 4 — Adapter

Campo	Contenido
Patrón y punto de dolor que resuelve	Adapter, responde a P-07. En el AS-IS conviven dos caminos de venta y solo uno está vivo: <code>Hacienda.vender_res</code> (<code>Hacienda.cs:182</code>), llamado desde <code>ResService.cs:79</code> . El genérico <code>vender</code> no se puede usar para una res porque pide <code>IInventario<Producto></code> y <code>Potrero</code> es <code>IInventario<Res></code> ; como <code>IInventario<T></code> declara <code>agregar(T)</code> y <code>retirar(T)</code> , el genérico es invariante y la conversión no compila.
Alternativas que evaluaron	No hacer nada: conservar <code>vender_res</code> y mantener los dos caminos. Se descarta porque toda regla de venta habría que escribirla dos veces. Cambiar <code>IInventario<T></code> a covariante en la biblioteca: descartada, obliga a partir la interfaz en dos y <code>Bib_Hacienda</code> estaba fuera de alcance por decisión del equipo. Duplicar el retiro y el registro de venta en <code>ResService</code>: descartada, mete lógica de dominio en la capa web.
Qué sale y qué entra	Sale: el método <code>Hacienda.vender_res</code> . Entra: <code>InventarioPotrero</code> (Adapter), que traduce <code>IInventario<Res></code> a <code>IInventario<Producto></code> . Vive en <code>p_mvchacienda/Servicios/</code> , no en la biblioteca, porque la incompatibilidad es de tipos y no de negocio.

Campo	Contenido
Cómo se relaciona	Lo construye <code>ResService.cs:130</code> en cada venta. Lo usa <code>Hacienda.vender(IInventario<Producto>, Producto, uint)</code> . Con Factory Method: el potrero que envuelve contiene reses creadas por los creadores concretos.
Impacto	Creadas 1, modificadas 1 (<code>ResService</code>), eliminadas 0 clases; desaparece el método <code>vender_res</code> . Anexo B: SC-1 vende un lácteo por el mismo <code>vender</code> genérico, sin un camino aparte.
Qué cuesta	La conversión de <code>Producto</code> a <code>Res</code> se verifica en ejecución y no en compilación: si alguien pasa un lácteo, <code>InventarioPotrero</code> lanza. Es un chequeo que antes hacía el compilador. Además obliga a implementar <code>agregar(Producto)</code> , que el flujo de venta nunca llama.
Origen	Propuesta de la herramienta, aceptada con la ubicación puesta por el equipo (B-05). B-11 registra la corrección posterior: lo declaramos patrón adoptado y no arreglo de compilación, porque está en el código entregado.

Actividad 4.1 · Matriz SOLID

Qué significa cada valor

Valor	Qué quiere decir
Refuerza	La estructura nueva facilita cumplir el principio comparada con el diseño anterior
Neutro	No cambia nada respecto al principio
Tensionado pero compensado	Hay una dependencia o una conversión que no se puede evitar, y está a la vista con algo que la contiene
Roto	El diseño incumple el principio. No usamos esta casilla

Matriz

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
Factory Method	Refuerza	Refuerza	Neutro	Refuerza	Tensionado pero compensado
Builder	Refuerza	Refuerza	Refuerza	Refuerza	Tensionado pero compensado
Observer	Refuerza	Refuerza	Neutro	Refuerza	Tensionado pero compensado
Adapter	Refuerza	Neutro	Tensionado pero compensado	Neutro	Neutro

Factory Method — P-01

SRP refuerza. Cada `ICreadorRes` sabe tres cosas y ninguna más: su categoría, a qué edades aplica y cómo instanciar su subtipo (`ICreadorRes.cs:8–18`, `CreadorTernero.cs:6–15`). El catálogo resuelve cuál toca (`CatalogoCreadoresRes.cs:38–45`) y `Hacienda` coordina el alta. Antes las tres cosas vivían mezcladas dentro de un switch.

OCP refuerza. Una categoría nueva se registra como un creador más. No hay ningún `case` que agregar. Lo probamos: el arnés mete un `CreadorVerificador` inventado para la prueba y funciona sin tocar `Hacienda` ni el catálogo (`HaciendaReto2.Verification/Program.cs:226–241`).

ISP refuerza. `ICreadorRes` pide lo justo para resolver y crear una res. Ningún creador tiene que implementar métodos vacíos.

DIP tensionado pero compensado. `Hacienda` recibe un catálogo concreto, no una abstracción. Lo compensan tres cosas. El catálogo sí depende de `IEnumerable<ICreadorRes>`. Qué creadores existen se decide en la raíz de composición (`p_mvcHacienda/Program.cs:72–78`), lejos de los consumidores. Y tiene un constructor alternativo, así que una prueba puede sustituirlo.

Builder — P-04

SRP refuerza. El director guarda la secuencia común (validar, numerar, registrar, resumir) y cada builder aporta solo lo suyo (`FabricadorVacunas.cs:49–105`, `BuilderBacteriana.cs:6–32`). Esa secuencia estaba escrita dos veces.

OCP refuerza. Una implementación nueva de `IVacunaBuilder` reutiliza `Crear` y `CrearLote` sin que el director cambie. El arnés lo demuestra con un `BuilderVerificador` que el director no conoce (`Program.cs:406–437`).

LSP refuerza. Los dos builders concretos cumplen el mismo contrato y siempre devuelven una `Vacuna` válida (`IVacunaBuilder.cs:10–29`). El director nunca necesita saber cuál tiene delante.

ISP refuerza. `IVacunaBuilder` expone lo que el director necesita para construir y para armar el resumen. Nada más.

DIP tensionado pero compensado. Las cuatro sobrecargas públicas instancian `BuilderBacteriana` o `BuilderViva` a mano (`FabricadorVacunas.cs:25–46`). Son entradas de compatibilidad: existen para que la API que ya consumía `Hacienda` siga igual. La parte extensible, que es la que importa, depende de `IVacunaBuilder` (:51–71).

Observer — P-03

SRP refuerza. Los publishers producen avisos y `RecolectorMensajes` los guarda mientras dura una operación (`RecolectorMensajes.cs:10–39`, `Hacienda.cs:229–305`). El recolector no sabe nada de pesos ni de esquemas de vacunación. Solo recibe cadenas.

OCP refuerza. Agregar otro consumidor de avisos no obliga a tocar `Hacienda`: se suscribe al evento que ya existe. Las suscripciones estables ocurren una sola vez, en el constructor (`Hacienda.cs:82–87`).

ISP refuerza. `IObservadorMensaje` tiene un método, `Recibir(string)`. No hay forma de hacerlo más pequeño.

DIP tensionado pero compensado. `Hacienda` guarda un `RecolectorMensajes` concreto, y los publishers siguen usando delegados propios en lugar de una abstracción común. La tensión no sale de la frontera de eventos: el recolector implementa `IObservadorMensaje`, la suscripción ocurre una vez, y el arnés comprueba que el número de handlers no crece y que una operación no lee los avisos de otra.

Adapter — P-07

SRP refuerza. `InventarioPotrero` hace una sola cosa: traducir `IInventario<Res>` a `IInventario<Producto>` (`InventarioPotrero.cs:10–43`). La venta la sigue coordinando `Hacienda` y el guardado lo sigue haciendo `ResService` (`ResService.cs:116–133`).

LSP tensionado pero compensado. Esta es la tensión más honesta del diseño. El contrato dice `Producto`, pero el adaptador solo funciona con `Res`: `ComoRes` rechaza cualquier otro producto y `contiene` devuelve falso. Un cliente que le pase un lácteo nota la diferencia. Lo que la contiene es que el adaptador se construye en un único sitio y solo para vender reses. No lo presentamos como un inventario de propósito general.

DIP neutro. El adaptador implementa la misma abstracción que `Hacienda` ya consumía (`IInventario.cs:6–11`, `InventarioPotrero.cs:4–10`). No agrega ni quita dependencias concretas, y mantiene la incompatibilidad de tipos fuera de la biblioteca de dominio.

Actividad 4.2 · Evidencia de comportamiento

Resultado

Ca so	Antes · 03-src/redisenado/HaciendaNEW	Después · 04-src/active
C0 1	OK El potrero P1 se a añadido a la hacienda. · potreros=1	OK El potrero P1 se a añadido a la hacienda. · potreros=1
C0 2	EXCEPTION Exception>Error inesperado en el metodo crear_potrero: Ya existe un potrero con el nombre 'p1'.	EXCEPTION Exception>Error inesperado en el metodo crear_potrero: Ya existe un potrero con el nombre 'p1'.
C0 5	OK La res Manchas ha sido añadida al potrero PC con exito. · tipo=Cebon	OK La res Manchas ha sido añadida al potrero PC con exito. · tipo=Cebon
C0 6	OK La res Toro ha sido añadida al potrero PN con exito. · tipo=Novillo	OK La res Toro ha sido añadida al potrero PN con exito. · tipo=Novillo
C0 7	OK La res 'Lola' ha sido alimentada, ahora pesa 101 kg.\n[Evento] La res 'Lola' tiene un peso 101, está en desnutrición. · peso=101	OK La res 'Lola' ha sido alimentada, ahora pesa 101 kg.\n[Evento] La res 'Lola' tiene un peso 101, está en desnutrición. · peso=101
C0 8	OK Vacuna bacteriana 'Bovina' del lote 'L1' agregada al inventario con éxito. Período de aplicación: 4 semanas. · vacunas=1	OK Vacuna bacteriana 'Bovina' del lote 'L1' agregada al inventario con éxito. Período de aplicación: 4 semanas. · vacunas=1
C0 9	EXCEPTION Exception>Error inesperado en el método crear_vacuna (bacteriana): Ya existe una vacuna con el lote 'l1' en el inventario	EXCEPTION Exception>Error inesperado en el método crear_vacuna (bacteriana): Ya existe una vacuna con el lote 'l1' en el inventario
C1 0	OK Vacuna viva 'Viral' del lote 'V1' agregada al inventario con éxito. Grado de atenuación: 10. · vacunas=1	OK Vacuna viva 'Viral' del lote 'V1' agregada al inventario con éxito. Grado de atenuación: 10. · vacunas=1
C1 1	OK Lote de vacunas bacterianas creado con éxito:\n- Nombre: {nombre}\n- Cantidad creada: 3 de 3\n- Lotes: LB-001 a LB-003\n- Período de aplicación: 4 semanas · vacunas=3	OK Lote de vacunas bacterianas creado con éxito:\n- Nombre: {nombre}\n- Cantidad creada: 3 de 3\n- Lotes: LB-001 a LB-003\n- Período de aplicación: 4 semanas · vacunas=3
C1 2	OK Lote de vacunas vivas creado con éxito:\n- Nombre: LoteV\n- Cantidad creada: 2 de 2\n- Lotes: LV-001 a LV-002\n- Grado de atenuación: 10 · vacunas=2	OK Lote de vacunas vivas creado con éxito:\n- Nombre: LoteV\n- Cantidad creada: 2 de 2\n- Lotes: LV-001 a LV-002\n- Grado de atenuación: 10 · vacunas=2
C1 3	EXCEPTION Exception>Error inesperado en el método crear_vacuna (bacteriana): el valor del periodo de aplicacion debe estar entre 2 y 4 semanas	EXCEPTION Exception>Error inesperado en el método crear_vacuna (bacteriana): el valor del periodo de aplicacion debe estar entre 2 y 4 semanas
C1 4	OK Vacuna aplicada correctamente a la res Lola. [Evento] La res 'Lola' aún no ha completado su esquema de vacunación. Bacterianas: 1, Vivas: 0 · inventario=0;aplicadas=1	OK Vacuna aplicada correctamente a la res Lola. [Evento] La res 'Lola' aún no ha completado su esquema de vacunación. Bacterianas: 1, Vivas: 0 · inventario=0;aplicadas=1
C1 5	EXCEPTION Exception>Error inesperado en el método crear_vacuna (bacteriana): La fecha de vencimiento debe ser posterior a la fecha de aplicación	EXCEPTION Exception>Error inesperado en el método crear_vacuna (bacteriana): La fecha de vencimiento debe ser posterior a la fecha de aplicación
C1 6	EXCEPTION Exception>Error inesperado en el metodo aplicar_vacuna: La vacuna 'Bovina' ya fue aplicada a la res 'Lola'.	EXCEPTION Exception>Error inesperado en el metodo aplicar_vacuna: La vacuna 'Bovina' ya fue aplicada a la res 'Lola'.
C1 7	OK Error inesperado en el metodo aplicar_vacuna: No se puede aplicar más vacunas bacterianas a la res 'Lola'. Ya tiene las 3 permitidas. · aplicadas=3	OK Error inesperado en el metodo aplicar_vacuna: No se puede aplicar más vacunas bacterianas a la res 'Lola'. Ya tiene las 3 permitidas. · aplicadas=3
C1 9	OK La res 'Gorda' ha sido alimentada, ahora pesa 410 kg.\n[Evento] La res 'Gorda' tiene un peso 410, apta para venta. · peso=410	OK La res 'Gorda' ha sido alimentada, ahora pesa 410 kg.\n[Evento] La res 'Gorda' tiene un peso 410, apta para venta. · peso=410

Caso	Antes · 03-src/redisenado/HaciendaNEW	Después · 04-src/active
C20	OK La res 'Lola' ha sido alimentada, ahora pesa 102 kg.\n[Evento] La res 'Lola' tiene un peso 102, está en desnutrición. · iguales=True	OK La res 'Lola' ha sido alimentada, ahora pesa 102 kg.\n[Evento] La res 'Lola' tiene un peso 102, está en desnutrición. · iguales=True

Casos nuevos que recorren lo que tocan los patrones

Caso	Patrón	Qué comprueba	Método
N01	Factory Method	Las fronteras 12, 13, 48 y 49 resuelven el creador correcto	FronterasDeEdadEntreCategorias :130-151
N02	Factory Method	Una categoría registrada después funciona sin tocar clientes	CatalogoAdmiteCategoriaNuevaSinTocarClientes :232-251
N03	Builder	Una variante que el Director no conoce reutiliza su secuencia	DirectorAdmiteVarianteNuevaSinModificarse :413-446
N04	Observer	Repetir la operación no acumula handlers	SuscripcionNoCreceConLasLlamadas :545-574
N05	Observer	La captura conserva el orden de emisión	CapturaConservaElOrdenDeEmision :575-609
N06	Observer	Una operación no lee los avisos de otra	CapturaAislaOperacionesEntreSi :610-637
N07	Adapter	La venta atraviesa InventarioPotrero, retira la res y registra la venta	VentaDeResUsaElAdapter :638-657
N08	SC-3	Cada res conserva su propia historia	HistoriaClinicaNoPuedeCompartirseEntreReses :760-771
N09	SC-3	Una vacuna se registra una vez y no se duplica como evento	VacunaAplicadaSeRegistraUnaVezEnHistoria :676-688
N10	Builder	El literal sin interpolar sobrevive al patrón	LoteBacterianoConservaLiteralNombreSinInterpolar :315-325

Verificaciones ejecutadas: 92
TODAS LAS VERIFICACIONES PASARON.

Actividad 5 · Registro de riesgos

I D	Riesgo (si ocurre X, entonces Y)	Prob	I m p	E x p	Qué hacen para evitarlo	Cómo se enteran de que está pasando	
R-01	Si se agrega una categoría de res y no se registra su <code>ICreadorRes</code> en <code>Program.cs:72-78</code> , entonces el alta o la carga de reses fallará porque el catálogo no encontrará un creador aplicable.	2	4	8	Registrar el creador en la raíz de composición y ejecutar las pruebas de frontera de edad y de categoría nueva.	Aparece Ninguna categoría de res cubre una edad... al crear o cargar una res, o <code>ParaCategoria</code> devuelve null.	
R-02	Si un <code>IVacunaBuilder</code> nuevo no conserva el contrato de variante y mensajes, entonces cambiará la salida observable de creación individual o por lote.	3	4	12	Verificar <code>Variante</code> , <code>VariantePlural</code> , <code>VarianteLote</code> , <code>DetalleEspecifico</code> y <code>NombreEnResumenDeLote</code> ; ejecutar las comparaciones de salida, incluido el literal <code>{nombre}</code> .	Falla una comparación de mensaje o aparece una etiqueta distinta a la esperada en la salida del sistema.	
R-03	Si una suscripción de <code>Observer</code> vuelve a ejecutarse dentro de una operación, entonces los handlers crecerán y los avisos se repetirán en una aplicación con <code>Hacienda</code> singleton.	3	4	12	Mantener las suscripciones en el constructor de <code>Hacienda</code> ; comprobar el número de handlers después de llamadas repetidas y verificar el aislamiento de cada captura.	El conteo de handlers supera uno o una operación devuelve avisos duplicados.	
R-04	Si <code>EventosClinicos.txt</code> contiene una línea inválida o un campo incompatible con el formato separado por `	` , entonces la carga omitirá ese evento y la historia clínica quedará incompleta.	3	5	15	Mantener el formato y <code>Sanitizar</code> ; probar guardado-carga y comparar cantidad y contenido de eventos antes y después de reiniciar.	Después de cargar, <code>HistoriaClinica.EventosClinicos.Count</code> o el contenido no coincide con lo guardado.
R-05	Si <code>InventarioPotrero</code> recibe un <code>Producto</code> que no es <code>Res</code> , entonces la venta fallará en tiempo de ejecución porque el <code>Adapter</code> no puede traducirlo al inventario del potrero.	2	3	6	Conservar la validación de <code>ComoRes</code> y usar el <code>Adapter</code> solo para ventas de reses; probar explícitamente el límite de tipos.	Se lanza <code>Unpotrero solo admite reses...</code> durante <code>agregar</code> o <code>retirar</code> .	

Referencias

- Factory Method: `04-src/active/p_mvchacienda/Program.cs:72-78` y `Servicios/PersistenciaService.cs:164-202`.
- Builder: `04-src/active/Bib_Hacienda/Bib_Hacienda/Interfaces/IVacunaBuilder.cs:10-29` y `Clases/FabricadorVacunas.cs:49-105`.
- Observer: `04-src/active/Bib_Hacienda/Bib_Hacienda/Clases/Hacienda.cs:82-87` y `Eventos/RecolectorMensajes.cs:10-39`.

- Historia clínica: 04-src/active/p_mvchacienda/Servicios/PersistenciaService.cs:420-509.
- Adapter: 04-src/active/p_mvchacienda/Servicios/InventarioPotrero.cs:10-43.

Actividad 6 · Vista para el negocio

Para la Dirección de Ingeniería y para quien aprueba el presupuesto.

En una frase

El sistema hace exactamente lo mismo que antes. Lo que cambió es cuánto cuesta pedirle algo nuevo.

Qué tocamos y qué no

Tocamos la forma en que las piezas internas del programa se llaman entre sí. No tocamos ninguna pantalla, ningún dato guardado, ninguna regla del negocio. Un empleado que use el sistema mañana no va a notar diferencia, salvo en un punto que explicamos más abajo y que hay que decidir.

Tampoco cambiamos la tecnología ni partimos el sistema en pedazos. Eso estaba fuera del encargo y sigue fuera.

Dónde se estaba yendo el tiempo

Medimos el costo contando cuántos sitios distintos del programa hay que abrir para hacer un cambio típico. No son estimaciones: son conteos.

Si el negocio pide...	Antes había que abrir	Y además
Una categoría nueva de animal, por ejemplo un toro reproductor	12 sitios del programa	5 de ellos con decisiones que hay que mantener iguales a mano
Un tipo nuevo de vacuna	17 sitios	El contrato interno cambia y arrastra a todo lo que lo usa
Un aviso nuevo, por ejemplo "animal listo para traslado"	8 sitios	Para saber quién escucha un aviso hay que leer el programa entero

El problema no es que sean muchos sitios. Es que **nada avisa si se olvida uno**. Encontramos un caso real de eso: al guardar los animales en disco, si el tipo no era conocido el sistema lo convertía en ternero, en silencio, y nadie se enteraba.

También encontramos 219 líneas escritas para controlar quién puede hacer qué, que **nunca se ejecutan**. Quien quiera cambiar permisos hoy va a editar el sitio equivocado y no va a ver ningún efecto.

Qué gana el negocio

Tiempo de respuesta a una solicitud. Después del cambio, agregar una categoría nueva de animal se hace registrándola en un solo sitio. Los otros once quedan enterados solos. Lo mismo con un aviso nuevo: un sitio en lugar de ocho.

Riesgo de romper algo. Antes, olvidar uno de los doce sitios producía un error que aparecía días después y en un lugar que no tenía nada que ver con la causa. Ahora, si falta el registro, el sistema lo dice de inmediato y con un mensaje que nombra el problema.

Una red de seguridad. Dejamos 92 comprobaciones automáticas y una comparación que pone lado a lado lo que el sistema respondía antes y lo que responde ahora. Cualquiera del equipo la corre en un minuto. Si alguien rompe algo, salta ahí y no en producción.

Qué costó

Más piezas. Pasamos de tener las decisiones concentradas en pocos sitios grandes a tenerlas repartidas en piezas pequeñas: doce piezas nuevas en total.

Eso tiene un precio real: leer el programa para entender cómo se crea un animal ahora exige abrir tres archivos en lugar de uno. A cambio, cambiarlo exige tocar uno en lugar de doce. Es un intercambio deliberado: pagamos en lectura para cobrar en modificación, porque leer se hace una vez y modificar se hace en cada solicitud.

Riesgos, en lenguaje de operación

Si pasa esto	Entonces	Cómo se enteran
Se agrega una categoría de animal y se olvida registrarla	El alta de animales de esa categoría falla	El sistema muestra un mensaje que dice que ninguna categoría cubre esa edad
Se agrega un tipo de vacuna sin respetar el formato de los mensajes	Los textos que ve el usuario cambian sin que nadie lo pida	Una de las 92 comprobaciones falla al correrla
El archivo donde se guarda la historia clínica se daña	Se pierden eventos clínicos sin aviso	Después de reiniciar, la historia de un animal tiene menos registros de los que se guardaron
Se intenta vender algo que no es un animal por la vía de los potreros	La venta falla en ese momento	El sistema muestra que un potrero solo admite reses

El más caro de la lista es el de la historia clínica, porque los datos perdidos no se recuperan. Por eso conviene revisar ese archivo cuando se haga una copia de seguridad.

Una cosa que cambió y hay que decidir

Al ordenar el sistema de avisos, **se perdió una advertencia**. Antes, al registrar un animal con poco peso, el sistema respondía con el mensaje de alta y además avisaba que estaba en desnutrición. Ahora responde solo con el mensaje de alta.

La información no se perdió: el peso sigue guardado y visible. Lo que se perdió es la advertencia automática en ese momento.

Recuperarla es trabajo de unas horas, pero implica cambiar lo que el sistema responde, y ese tipo de cambio necesita autorización. **No lo hicimos por nuestra cuenta**. Lo dejamos anotado y lo traemos aquí para que se decida.

Qué necesitamos del negocio

1. **Una decisión sobre esa advertencia.** ¿Se restaura o se deja como está? Si se restaura, necesitamos la autorización para cambiar lo que el sistema responde al registrar un animal. 2. **Una decisión sobre los permisos.** Hay 219 líneas escritas para controlar quién puede vender, quién puede vacunar y quién solo puede consultar, y no están conectadas. Antes de conectarlas hay que saber si el negocio quiere que el sistema empiece a negar operaciones que hoy permite, porque el día que se conecten habrá gente que deje de poder hacer cosas. 3. **Una persona que valide las salidas.** Cuando cambiemos algo que el usuario ve, necesitamos a alguien del negocio que confirme que el texto nuevo dice lo que debe decir.

Qué pasa si no se sigue

Nada se rompe mañana. El costo es acumulativo y ya lo estamos pagando.

Cada solicitud nueva que llegue sobre las partes que no tocamos seguirá costando lo mismo que antes: abrir muchos sitios y confiar en que nadie olvide uno. Y esa confianza ya falló al menos una vez, en el caso del animal que se convertía en ternero sin avisar.

Las 219 líneas de permisos que no corren seguirán ahí, y cada persona nueva que entre al equipo va a perder medio día entendiéndolo que no hacen nada.

Actividad 6 · Vista para el equipo

Para quien entra al equipo y tiene que cambiar algo sin romper nada. No hace falta haber estado en ninguna reunión.

Todas las rutas salen de `05-reto2-patrones/04-src/active/`.

Dónde se ensambla todo

`p_mvchacienda/Program.cs`. Ese archivo es la única raíz de composición: es el sitio donde se decide qué implementación concreta usa cada cosa. Ningún otro archivo debería instanciar colaboradores del dominio.

Tres bloques importan:

```
// :74-78 los creadores de res y su catálogo
builder.Services.AddSingleton<ICreadorRes, CreadorTernero>();
builder.Services.AddSingleton<ICreadorRes, CreadorCebon>();
builder.Services.AddSingleton<ICreadorRes, CreadorNovillo>();
builder.Services.AddSingleton<CatalogoCreadoresRes>(sp =>
    new CatalogoCreadoresRes(sp.GetServices<ICreadorRes>()));

// :87 el puerto de persistencia de eventos clínicos
// :106-108 el orden de carga al arrancar
```

Si algo no aparece cuando el sistema arranca, este archivo es el primer sitio donde mirar.

Qué patrones hay y dónde vive cada uno

Patrón	Papel	Archivos
Factory Method	Decide qué subtipo de Res se instancia	<code>Bib_Hacienda/Interfaces/ICreadorRes.cs</code> , <code>Clases/Creacion/</code> (catálogo y tres creadores)
Builder	Separa la secuencia de creación de vacunas de lo que cambia por variante	<code>Interfaces/IVacunaBuilder.cs</code> , <code>Clases/Construccion/</code> , con <code>Clases/FabricadorVacunas.cs</code> como Director
Observer	Fija quién escucha los avisos y por cuánto tiempo	<code>Interfaces/IObservadorMensaje.cs</code> , <code>Eventos/RecolectorMensajes.cs</code> , <code>Eventos/CapturaMensajes.cs</code>
Adapter	Deja que un potrero pase por el contrato genérico de inventario	<code>p_mvchacienda/Servicios/InventarioPotrero.cs</code>

Cómo se relacionan entre sí: solo hay una conexión real. El potrero que envuelve el Adapter contiene reses que creó el Factory Method. Builder y Observer no se tocan con nadie.

`HistoriaClinica` y `EventoClinico` no son un patrón. Son composición de dominio y salieron de la solicitud SC-3.

La guía de dónde tocar

Quiero...	Crear	Modificar	No tocar
Agregar una categoría de res (por ejemplo un toro reproductor)	Una clase <code>Res</code> hija y su <code>ICreadorRes</code> en <code>Clases/Creacion/</code>	<code>Program.cs:74-78</code> para registrar el creador nuevo	<code>Potrero</code> , <code>Hacienda</code> , <code>PersistenciaService</code> . Si te ves modificándolos, el creador está mal hecho

Quiero...	Crear	Modificar	No tocar
Agregar un tipo de vacuna	Una clase <code>Vacuna</code> hija y su <code>IVacunaBuilder</code> en <code>Clases/Construccion/</code>	<code>enum/TipoVacuna.cs</code> y <code>Interfaces/ICreacionVacuna.cs</code> , que sigue publicando cuatro firmas	<code>El Director FabricadorVacunas</code> . Si tienes que tocarlo, el builder no está cumpliendo el contrato
SC-1 · vender un producto derivado (lácteo, carne, piel)	Un <code>IInventario<Producto></code> para ese producto, o reusar <code>InventarioCarnes</code> / <code>InventarioLacteos</code> / <code>InventarioPiel</code> , que existen pero nadie instancia	<code>Program.cs</code> para registrarlo y <code>VentaService</code> para exponerlo	<code>Hacienda.vender</code> , que ya es genérico. No hace falta una vía nueva de venta
SC-2 · poner chips de geolocalización	Un publisher para el aviso de posición, en <code>Eventos/</code>	<code>Res</code> para el identificador del chip, <code>PersistenciaService</code> para guardarlo, y <code>Hacienda.cs:84-86</code> para suscribir el publisher nuevo	El resto de los publishers y <code>RecolectorMensajes</code> . Un observador nuevo se suscribe, no se modifica
SC-3 · ampliar la historia clínica	Nada, si es un evento más. <code>EventoClinico</code> ya acepta fecha, concepto y observación	<code>Views/Res/HistoriaClinica.cshtml</code> si hay que mostrarlo distinto	<code>HistoriaClinica.RegistrarVacuna</code> . Las vacunas se registran como vacunas, no como eventos
Agregar un aviso nuevo	Un publisher en <code>Eventos/</code>	<code>Hacienda.cs:84-86</code> para suscribirlo al recolector	Los publishers existentes. Y ojo: emitir suscribir hace que el aviso se pierda en silencio
Cambiar dónde se guardan los datos	Una implementación nueva de los puertos de persistencia	<code>Program.cs</code> para registrarla	<code>PersistenciaService</code> , que se reemplaza entero. Los puertos viven en <code>Bib_Hacienda/Interfaces/</code> y en <code>p_mvcHacienda/Servicios/IPersistenciaEventosClinicos.cs</code>

Reglas que no se deben romper

El catálogo no lleva condicionales. `CatalogoCreadoresRes.ParaEdad` resuelve preguntando a cada creador si aplica. El día que alguien meta un `switch` ahí, volvemos al problema que este trabajo vino a resolver: cada categoría nueva obligaría a modificar el catálogo.

Las suscripciones van en el constructor de `Hacienda`, nunca dentro de un método. `Hacienda` vive como singleton mientras dure el proceso. Un `+=` dentro de un método agrega un handler en cada llamada y los avisos empiezan a duplicarse. Así estaba antes y así se rompía.

El literal roto de `FabricadorVacunas.cs:86` se conserva a propósito. El resumen del lote bacteriano imprime `- Nombre: {nombre}` sin interpolar, porque falta un `$`. Es salida observable y está congelada. `BuilderBacteriana.NombreEnResumenDeLote` devuelve `"{nombre}"` para preservarlo. No lo arregles sin autorización: hay una prueba que falla si lo haces, y está puesta a propósito.

El Adapter solo sirve para vender reses. `InventarioPotrero` acepta `Producto` por firma pero lanza con cualquier cosa que no sea `Res`. Se construye en un único sitio, `ResService.cs:130`. No lo uses como inventario de propósito general.

Antes de dar por bueno un cambio, corre las dos verificaciones.

```
dotnet run --project 04-src/HaciendaReto2.Verification # 92 controles
cd 04-verificacion/caracterizacion # salidas antes/después
dotnet run --project Caracterizacion.AsIs > SALIDA-ASIS.txt
dotnet run --project Caracterizacion.ToBe > SALIDA-TOBE.txt
diff SALIDA-ASIS.txt SALIDA-TOBE.txt
```

El `diff` debe mostrar C03, C04 y C18. Ni una línea más.

Deuda pendiente

Tres avisos que no llegan a nadie. `PublisherPotreroMitad` y `PublisherPotreroLleno` se instancian en `Potrero.cs:23-24`, y `PublisherVacunaVencida` en `Hacienda.cs:46`. Los tres emiten y nadie los escucha. La consecuencia visible está registrada como C03: al dar de alta una res desnutrida ya no aparece la advertencia. Cerrarlo exige decidir qué devuelve `anadir_res_potrero`, que hoy no concatena eventos.

El contrato de vacunas sigue con cuatro firmas. `ICreacionVacuna` publica cuatro sobrecargas de `crear_vacuna`. El Builder eliminó el cuerpo duplicado, no el contrato. Un tipo nuevo de vacuna todavía obliga a tocar esa interfaz.

El Adapter verifica el tipo en ejecución. `InventarioPotrero` convierte de `Producto` a `Res` con un chequeo que antes hacía el compilador. Arreglarlo de raíz exige partir `IInventario<T>`, y eso toca la biblioteca de dominio.

Doscientas diecinueve líneas de control de acceso que no corren. `Autenticacion` y `InterceptorAutenticacion` existen, tienen las reglas de permisos por rol y nadie las instancia. Quien vaya a tocar permisos va a editar el archivo equivocado. Está sin conectar a propósito: hacerlo empezaría a denegar operaciones que hoy pasan, y eso necesita autorización del negocio.

Tres inventarios idénticos. `InventarioCarnes`, `InventarioLacteos` e `InventarioPielles` son la misma clase copiada tres veces, 64 líneas cada una, y ninguna se instancia. Si llega SC-1, es el primer sitio a mirar y probablemente a unificar.

Por dónde empezar a leer

1. `Program.cs`, para ver cómo se arma el sistema. 2. `Clases/Hacienda.cs`, que coordina casi todo. 3. La guía de arriba, para el cambio concreto que vengas a hacer.

Los diagramas están en `03-diseno/diagramas/`. Tienen dos capas: apagando T0-BE se ve cómo estaba antes, encendiendo las dos se ve cómo quedó.